

浙江大学

本科实验报告

课程名称：计算机组成与设计

姓 名：张序鸿

学 院：竺可桢学院

专 业：计算机科学与技术

学 号：3230103265

指导教师：杨莹春

2024 年 12 月 13 日

浙江大学实验报告

课程名称: 计算机组成与设计 实验类型: 综合

实验项目名称: 流水线处理器—Ex、Mem、WB 设计与集成

学生姓名: 张序鸿 专业: 混合班 学号: 3230103265

指导老师: 杨莹春、潘之杰 助教: 马致远

实验地点: 东 4-509 实验日期: 2024 年 12 月 11 日

一、实验目的和要求

- 1.理解流水线 CPU 的基本原理和组织结构
- 2.掌握五级流水线的工作过程和设计方法
- 3.理解流水线执行、存储器访问、写回的原理
- 4.设计流水线测试程序

二、实验内容和原理

内容:

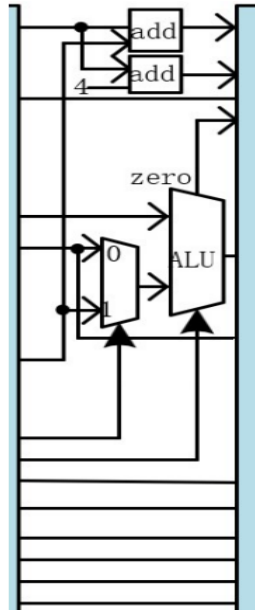
任务一: 设计执行 (Ex)、存储器访问 (Mem)、写回 (WB) 模块

- 设计执行模块, 替换 OExp05-2 的执行模块并完成集成
- 设计访存模块, 替换 OExp05-2 的访存模块并完成集成
- 设计写回模块, 替换 OExp05-2 的写回模块并完成集成

任务二: 设计流水线测试方案并完成测试

原理:

2.1 执行 Ex 部件



Ex : 执行阶段涉及运算单元 (ALU) 它获取操作数并完成指定的算数运算或逻辑运算。

Ex_reg_Mem: 暂存运算结果和控制信号, 以待下一级使用。

(1) 32 位多路选择器:

本质是一个 32 位二选一的多路器, 用于选择 ALU 的第二个操作数, 当 $ALUSrc_B = 0$, 选择寄存器堆数据 2,; 当 $ALUSrc_B = 1$, 选择立即数。

```
module MUX2T1_32(  
    input [31:0] I0,  
    input [31:0] I1,  
    input s, // 此处输入 s = ALUSrc_B  
    output [31:0] o  
);  
  
    assign o = s ? I1 : I0;  
endmodule
```

(2) 加法器 (add):

本质是一个 32 位输入的加法器, 一个输出计算 $PC+4$ 、一个输出计算 $PC+Imm$ 。

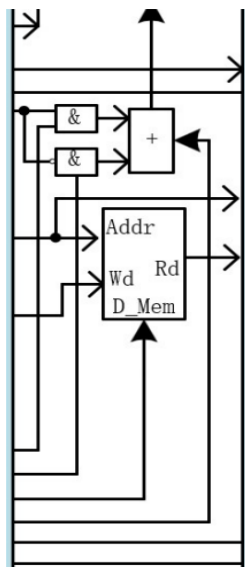
```
`timescale 1ns / 1ps  
module add32(  
    input [31:0] a,  
    input [31:0] b,  
    output [31:0] c  
);  
  
    assign c = a + b;  
endmodule
```

(3) ALU:

本质是一个二输入 32 位的运算器，实现算数运算和逻辑运算

```
module ALU(  
    input [31:0] A,  
    input [3:0] ALU_operation, // 4 位采用了指令拓展  
    input [31:0] B,  
    output reg [31:0] res,  
    output zero  
);  
always @ (*)  
    case (ALU_operation)  
        4'b0000: res = A & B; //and  
        4'b0001: res = A | B; //or  
        4'b0010: res = A + B; //add  
        4'b1100: res = A ^ B; //xor  
        4'b1101: res = A >> B; //sr1  
        4'b0110: res = A - B; //sub  
        4'b0111: res = ($signed(A)<$signed(B)) ? 32'h1 : 32'h0; //slt  
        4'b1111: res = ($signed(A) >> B); //sra  
        4'b1110: res = A << B; //sll  
        4'b1001: res = (A < B) ? 32'h1 : 32'h0; //sltu  
        default: res = 32'hx;  
    endcase  
  
    assign zero = (res==0)? 1: 0;  
endmodule
```

2.2 访存 Mem 部件



Mem：存储器访问阶段涉及数据存储器 (D_Mem)，Load\Store 指令对数据存储器进行读或写。

Mem_reg_WB：暂存存储器结果 和控制信号，以待下一级使用

(1) PCSrc 控制:

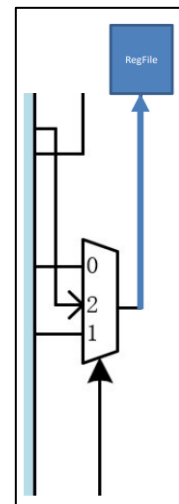
两输入与门和四输入或门 (beq、bne、jal、jalr) 组成, 用于判断分支和跳转是否发生, 生成 PCSrc 选择控制 PC 值。

(2) 数据储存器 (D_Mem):

本质是一个 RAM (可读可写), LOAD、STORE 指令的操作对象。同样的, 这只在构建 SOC 系统是用到。

2.3 写回 WB 部件

写回阶段涉及寄存器堆 (RegisterFiles), 将 ALU 的运算结果、存储器输出结果、PC+4 写回到寄存器堆。



(1) 4 路多路选择器:

本质是一个 32 位四选一的多路器, 用于选择写回寄存器堆的数据。

- 当 MemtoReg=0; 选择 ALU 输出,
- 当 MemtoReg=1; 选择存储器 (Memory) 输出
- 当 MemtoReg=2; 选择 PC+4。

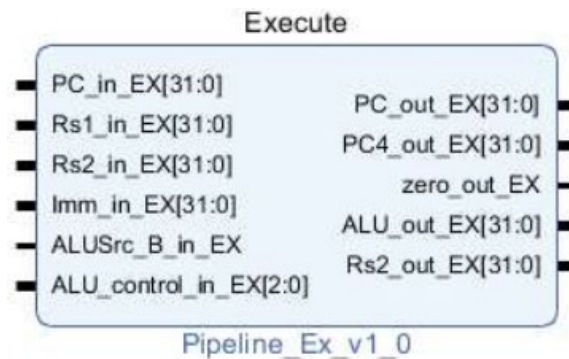
```
module MUX4T1_32(  
    input [31:0] I0,  
    input [31:0] I1,  
    input [31:0] I2,  
    input [31:0] I3,  
    input [1:0] s,  
    output reg [31:0] o  
);  
always @ (*) begin  
    case(s)  
        2'b00: o <= I0;  
        2'b01: o <= I1;  
        2'b10: o <= I2;  
        2'b11: o <= I3;  
    endcase  
end  
endmodule
```

三、实验过程和数据记录

3.1 设计实现执行（Ex）模块：

① Pipeline_Ex :

PipelineCPU 第三阶段，执行指令所指定的操作（ALU 计算、对 PC 进行 +4 或加立即数处理）。



```
`timescale 1ns / 1ps
module Pipeline_Ex(
    input[31:0] PC_in_EX, //PC 输入
    input[31:0] Rs1_in_EX, //操作数 1 输入
    input[31:0] Rs2_in_EX, //操作数 2 输入
    input[31:0] Imm_in_EX , //立即数输入
    input ALUSrc_B_in_EX , //ALU B 选择
    input[3:0] ALU_control_in_EX, //ALU 选择控制
    output [31:0] PC_out_EX, //PC 输出
    output [31:0] PC4_out_EX, //PC+4 输出
    output zero_out_EX, //ALU 判 0 输出
    output [31:0] ALU_out_EX, //ALU 计算输出
    output [31:0] Rs2_out_EX //操作数 2 输出
);

wire[31:0] mux_out;
assign Rs2_out_EX = Rs2_in_EX;

add32 add_32_0(
    .a(PC_in_EX),
    .b(Imm_in_EX),
    .c(PC_out_EX)
);

add32 add_32_1(
    .a(32'h4),
```

```

        .b(PC_in_EX),
        .c(PC4_out_EX)
    );

    ALU ALU(
        .A(Rs1_in_EX),
        .ALU_operation(ALU_control_in_EX),
        .B(mux_out),
        .res(ALU_out_EX),
        .zero(zero_out_EX)
    );

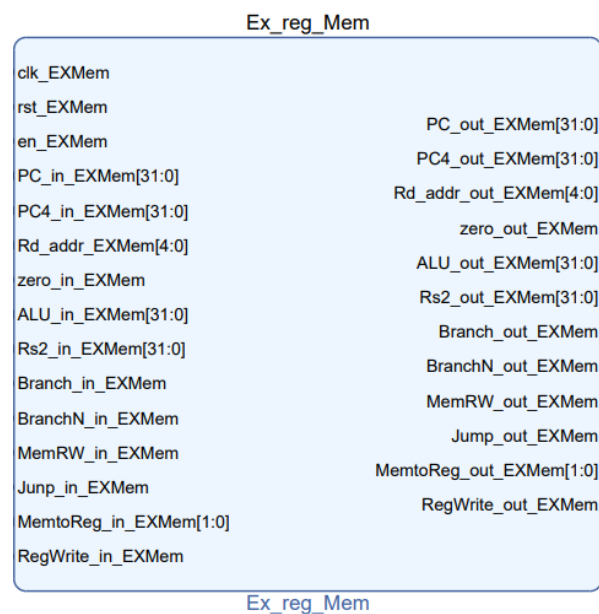
    MUX2T1_32 MUX2T1_32_0(
        .I0(Rs2_in_EX),
        .I1(Imm_in_EX),
        .s(ALUSrc_B_in_EX),
        .o(mux_out)
    );

endmodule

```

② Ex_reg_Mem :

PipelineCPU 执行和访存之间的寄存器，存储 ALU 数据和控制信号。寄存 EX 级的输出指令，分割 EX 级和 MEM 级的指令或控制信号，防止相互干扰，在 EX 级执行结束时将指令的控制信号传递至下一级。



```

`timescale 1ns / 1ps

module Ex_reg_Mem(
    input clk_EXMem,
    input rst_EXMem,
    input en_EXMem,
    input [31:0]PC_in_EXMem,
    input [31:0]PC4_in_EXMem,
    input [4:0]Rd_addr_EXMem,
    input zero_in_EXMem,
    input [31:0]ALU_in_EXMem,
    input [31:0]Rs2_in_EXMem,
    input Branch_in_EXMem,
    input BranchN_in_EXMem,
    input MemRW_in_EXMem,
    input [1:0]Jump_in_EXMem,
    input [1:0]MemtoReg_in_EXMem,
    input RegWrite_in_EXMem,

    output reg[31:0]PC_out_EXMem,
    output reg[31:0]PC4_out_EXMem,
    output reg[4:0]Rd_addr_out_EXMem,
    output reg zero_out_EXMem,
    output reg[31:0]ALU_out_EXMem,
    output reg[31:0]Rs2_out_EXMem,
    output reg Branch_out_EXMem,
    output reg BranchN_out_EXMem,
    output reg MemRW_out_EXMem,
    output reg[1:0]Jump_out_EXMem,
    output reg[1:0]MemtoReg_out_EXMem,
    output reg RegWrite_out_EXMem
);

always @(posedge clk_EXMem or posedge rst_EXMem)
begin
    if(rst_EXMem == 1'b1) begin
        PC_out_EXMem <= 32'h0;
        PC4_out_EXMem <= 32'h0;
        Rd_addr_out_EXMem <= 5'h0;
        zero_out_EXMem <= 1'h0;
        ALU_out_EXMem <= 32'h0;
        Rs2_out_EXMem <= 32'h0;
        Branch_out_EXMem <= 1'h0;
        BranchN_out_EXMem <= 1'h0;
    end
end

```



```

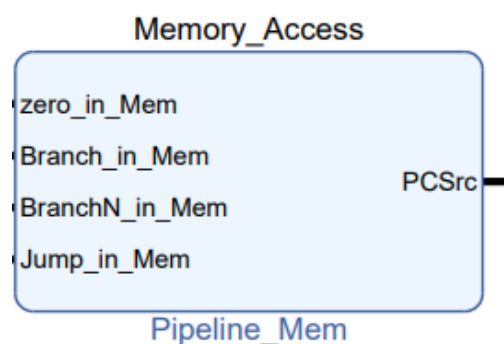
        MemRW_out_EXMem <= 1'h0;
        Jump_out_EXMem <= 2'h0;
        MemtoReg_out_EXMem <= 2'h0;
        RegWrite_out_EXMem <= 1'h0;
    end
    else if(en_EXMem == 1'b1) begin
        PC_out_EXMem <= PC_in_EXMem;
        PC4_out_EXMem <= PC4_in_EXMem;
        Rd_addr_out_EXMem <= Rd_addr_EXMem;
        zero_out_EXMem <= zero_in_EXMem;
        ALU_out_EXMem <= ALU_in_EXMem;
        Rs2_out_EXMem <= Rs2_in_EXMem;
        Branch_out_EXMem <= Branch_in_EXMem;
        BranchN_out_EXMem <= BranchN_in_EXMem;
        MemRW_out_EXMem <= MemRW_in_EXMem;
        Jump_out_EXMem <= Jump_in_EXMem;
        MemtoReg_out_EXMem <= MemtoReg_in_EXMem;
        RegWrite_out_EXMem <= RegWrite_in_EXMem;
    end
end
endmodule

```

3.2 设计实现访存（Mem）模块：

① Pipeline_Mem :

PipelineCPU 第四阶段，访问存储器，从 Memory 中读出数据或写入 Memory 的过程。



此处的输出 PCSrc 用于判断是否发生了指令跳转（如 beq、bne、jal 等），并作为 Pipeline_IF 的输入信号，从而获得下一条指令的地址。

```

`timescale 1ns / 1ps

module Pipeline_Mem(
    input zero_in_Mem,
    input Branch_in_Mem,
    input BranchN_in_Mem,
    input [1:0]Jump_in_Mem,

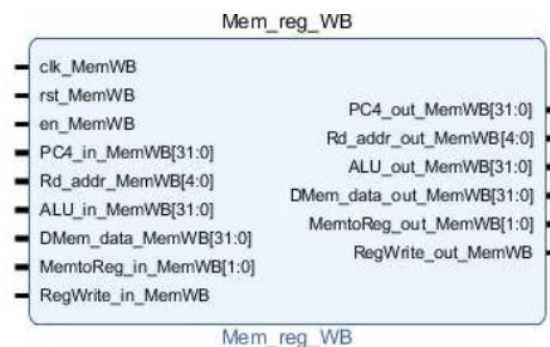
    output PCSrc
);

    assign PCSrc = (Branch_in_Mem & zero_in_Mem) | (BranchN_in_Mem &
(~zero_in_Mem)) | Jump_in_Mem[0] | Jump_in_Mem[1];
endmodule

```

② Mem_reg_WB :

PipelineCPU 访存和写回间的寄存器，其中存储了 ALU 数据和存储器相关数据。寄存 Mem 级的输出指令，以及输出数据，传递给写回阶段和寄存器堆。



```

`timescale 1ns / 1ps
module Mem_reg_WB(
    input clk_MemWB,
    input rst_MemWB, //寄存器复位
    input en_MemWB, //寄存器使能
    input[31:0] PC4_in_MemWB, //PC+4 输入
    input[4:0] Rd_addr_MemWB, //写目的地址输入
    input[31:0] ALU_in_MemWB, //ALU 输入
    input[31:0] DMem_data_MemWB, //存储器数据输入
    input[1:0] MemtoReg_in_MemWB, //写回
    input RegWrite_in_MemWB, //寄存器堆读写
    output reg[31:0] PC4_out_MemWB, //PC+4 输出
    output reg[4:0] Rd_addr_out_MemWB, //写目的地址输出
    output reg[31:0] ALU_out_MemWB, //ALU 输出
    output reg[31:0] DMem_data_out_MemWB, //存储器数据输出

```

```

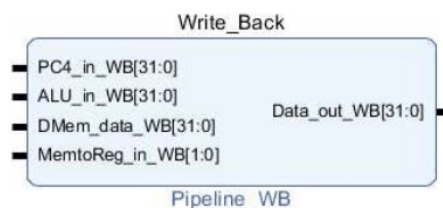
output reg[1:0] MemtoReg_out_MemWB, //写回
output reg RegWrite_out_MemWB //寄存器堆读写;
);
always @(posedge clk_MemWB or posedge rst_MemWB) begin
    if(rst_MemWB == 1'b1) begin
        PC4_out_MemWB <= 32'h0;
        Rd_addr_out_MemWB <= 5'h0;
        ALU_out_MemWB <= 32'h0;
        DMem_data_out_MemWB <= 32'h0;
        MemtoReg_out_MemWB <= 2'h0;
        RegWrite_out_MemWB <= 1'h0;
    end
    else if(en_MemWB == 1'b1) begin
        PC4_out_MemWB <= PC4_in_MemWB;
        Rd_addr_out_MemWB <= Rd_addr_MemWB;
        ALU_out_MemWB <= ALU_in_MemWB;
        DMem_data_out_MemWB <= DMem_data_MemWB;
        MemtoReg_out_MemWB <= MemtoReg_in_MemWB;
        RegWrite_out_MemWB <= RegWrite_in_MemWB;
    end
end
end
endmodule

```

3.3 设计实现写回（WB）模块：

① Pipeline_WB :

PipelineCPU 第五阶段，将结果写回寄存器堆的操作。如果是普通运算指令，该结果值来源于 Ex 阶段计算的结果；如果是 LOAD 指令，该结果来源于 Mem 阶段从存储器读取出来的数据；如果是跳转 (jal) 指令，该结果来源于 PC+4



```

module Pipeline_WB(
    input[31:0] PC4_in_WB, //PC+4 输入
    input[31:0] ALU_in_WB, //ALU 结果输出
    input[31:0] DMem_data_WB, //存储器数据输入
    input[1:0] MemtoReg_in_WB, //写回选择控制
    output [31:0] Data_out_WB //写回数据输出
);

```

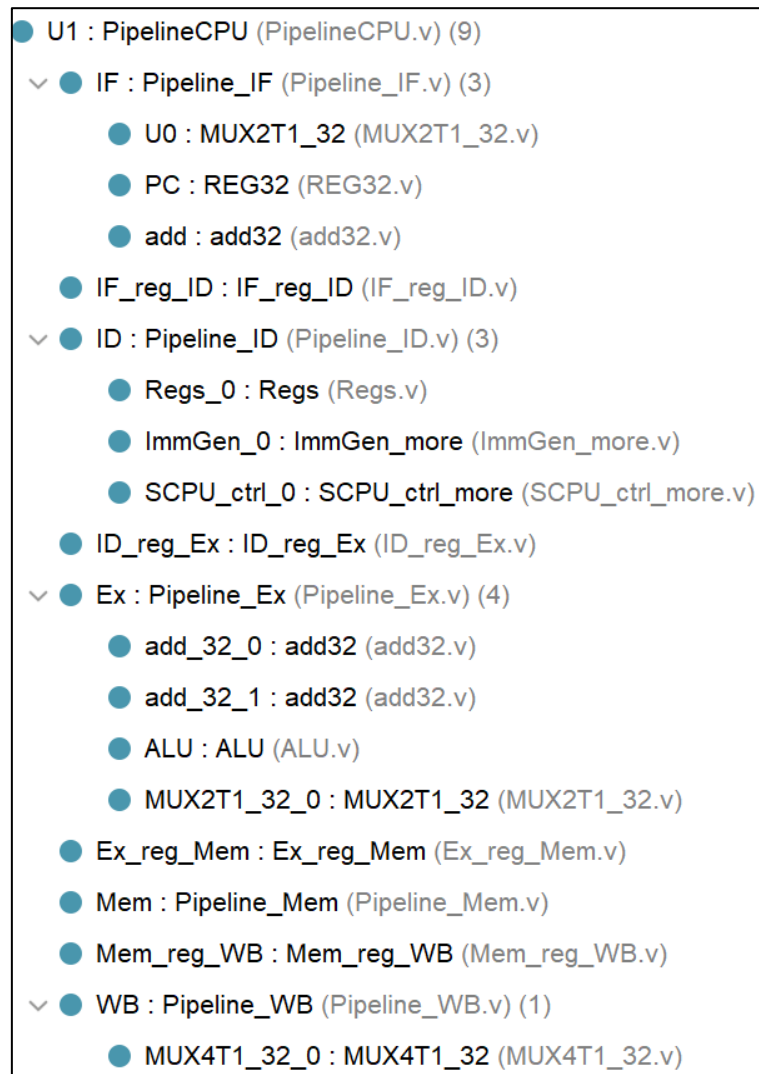
```

MUX4T1_32 MUX4T1_32_0(
    .s(MemtoReg_in_WB),
    .I0(ALU_in_WB),
    .I1(DMem_data_WB),
    .I2(PC4_in_WB),
    .I3(PC4_in_WB),
    .o(Data_out_WB)
);
endmodule

```

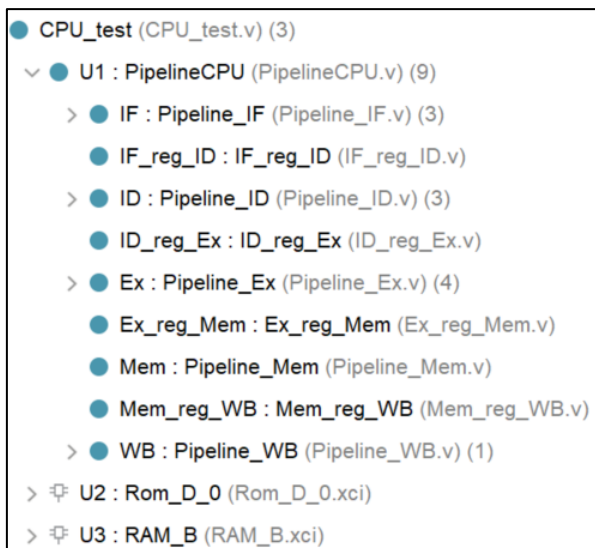
3.2 模块替换、集成流水线 CPU:

利用 Lab5-1 设计的 Pipeline_CPU.v 文件，将提供的流水线各阶段文件（.edf 文件）替换为 Lab5-2~3 实现的模块，以下为替换后 Pipeline_CPU 的工程结构：



3.3.1 搭建仿真平台

同样的，利用 PipelineCPU、RAM、ROM 可以搭建如下图的仿真平台（需要调整 PipelineCPU 模块的输入信号），初始化 RAM 的数据（D_mem.coe），初始化 ROM 的指令（P_mem.coe），查看 CPU 数据的输入和输出，以及底层的包括寄存器在内的各模块的状态。



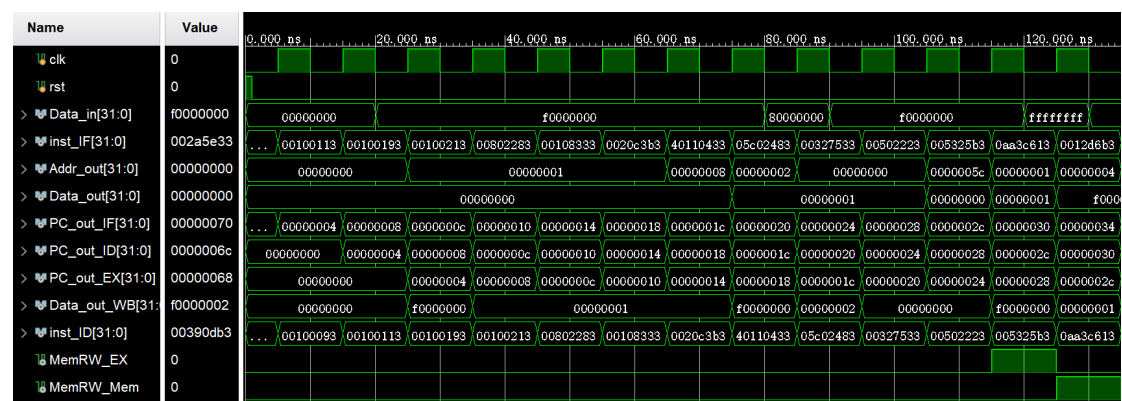
```
module CPU_test(
    input clk,
    input rst
);
    wire [31:0] Data_in, inst_IF, Addr_out, Data_out, PC_out_IF,
    PC_out_ID, PC_out_EX, Data_out_WB, inst_ID;
    wire MemRW_EX, MemRW_Mem;

    PipelineCPU U1(
        .clk(clk),
        .rst(rst),
        .Data_in(Data_in),
        .inst_IF(inst_IF),
        .PC_out_IF(PC_out_IF),
        .PC_out_ID(PC_out_ID),
        .inst_ID(inst_ID),
        .PC_out_IDEX(PC_out_EX),
        .MemRW_EX(MemRW_EX),
        .MemRW_Mem(MemRW_Mem),
        .Addr_out(Addr_out),
        .Data_out(Data_out),
        .Data_out_WB(Data_out_WB)
    );
```

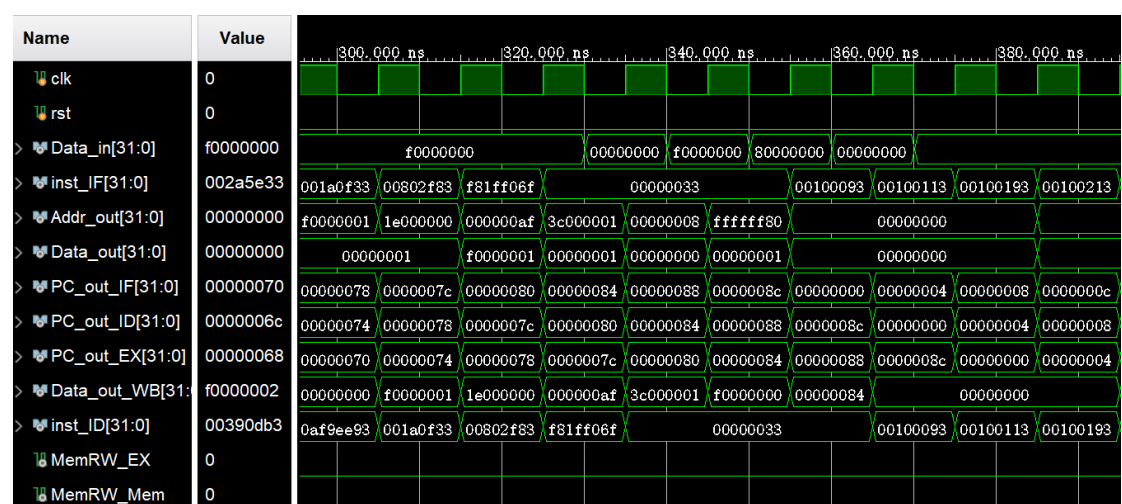
```

Rom_D_0 U2(
    .a(PC_out_IF[11:2]),
    .spo(inst_IF)
);
RAM_B U3(
    .addr_a(Addr_out[11:2]),
    .clk_a(~clk),
    .dina(Data_out),
    .douta(Data_in),
    .wea(MemRW_Mem)
);
endmodule

```



可以看到，PC_out_IF、PC_out_ID、PC_out_Ex 能够正常指定下一条指令，实现了每一时钟周期读取一条新指令的 Pipeline。



在指令集最后 PC = 0x80 时执行 jal x0 -128，可以看到经过 4 个时钟周期的缓冲（Pipeline_Mem 部件生成 PCSrc）后，PC 地址回到 0x0，从而实现 jal 跳转。

3.3.2 DEMO 物理测试

继承替换 CPU 及子模块后，下载.bit 文件，使用 DEMO 程序进行物理验证。
本实验由于指令种类多样并且指令较为复杂，通过记录 VGA 中的数据变化，设计了测试记录表格（见实验数据分析）。

- ROM 中填充指令如下：

P_Mem（无冒险）

```
1 memory_initialization_radix=16;
2 memory_initialization_vector=
3 00100093,
4 00100113,
5 00100193,
6 00100213,
7 00802283,
8 00108333,
9 0020C3B3,
10 40110433,
11 05C02483,
12 00327533,
13 00502223,
14 00532583,
15 0AA3C613,
16 0012D6B3,
17 00147713,
18 0034E7B3,
19 00A50833,
20 0085C8B3,
21 00402903,
22 004629B3,
23 0016DA13,
24 00677AB3,
25 40128B33,
26 00150B93,
27 00986C33,
28 00B9CCB3,
29 0FFA7D13,
30 00390DB3,
31 002A5E33,
32 0AF9EE93,
33 001A0F33,
34 00802F83,
35 F81FF06F,
36 00000033,
37 00000033,
38 00000033;
```

h_Mem（有冒险）

```
1 memory_initialization_radix=16;
2 memory_initialization_vector=
3 00100093,
4 00100113,
5 00100193,
6 00100213,
7 00802283,
8 00128333,
9 0020C3B3,
10 40708433,
11 05C02483,
12 00327533,
13 00502223,
14 00532583,
15 0AA3C613,
16 00818663,
17 00000013,
18 00000033,
19 0012D6B3,
20 00147713,
21 0034E7B3,
22 00A50833,
23 0085C8B3,
24 00402903,
25 004629B3,
26 0016DA13,
27 00677AB3,
28 01071463,
29 00000013,
30 40128B33,
31 00150B93,
32 00986C33,
33 00B9CCB3,
34 0FFA7D13,
35 00390DB3,
36 002A5E33,
37 0AF9EE93,
38 001A0F33,
39 00802F83,
40 F6DFF06F,
41 00000033,
42 00000033,
43 00000033;
```

- RAM 中储存的数据（D_mem.coe）如下：

```
memory_initialization_radix=16;
memory_initialization_vector=
F0000000, 000002AB, 80000000, 0000003F, 00000001, FFF70000,
0000FFFF, 80000000, 00000000, 11111111, 22222222, 33333333,
44444444, 55555555, 66666666, 77777777, 88888888, 99999999,
aaaaaaaa, bbbbbbbb, cccccccc, dddddddd, eeeeeeee, FFFFFFFF,
557EF7E0, D7BDFBD9, D7DBFDB9, DFCFFCFB, DFCFBFFF, F7F3DFFF,
FFFFDF3D, FFFF9DB9, FFFBFCFB, DFCFFCFB, DFCFBFFF, D7DB9FFF,
D7DBFDB9, D7BDFBD9, FFFF07E0, 007E0FFF, 03bdf020, 03def820,
08002300
```

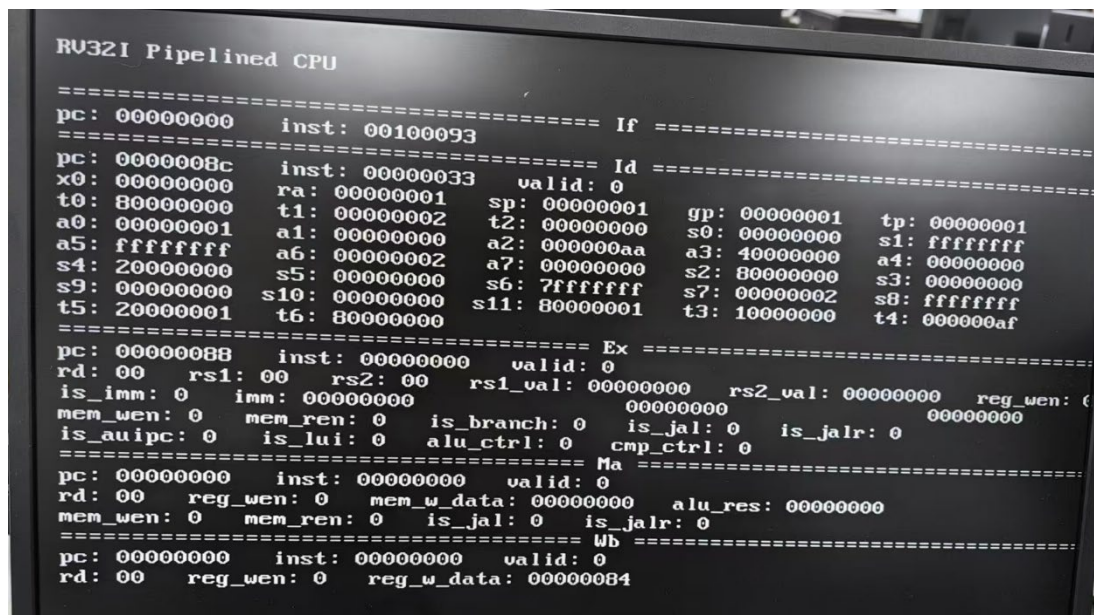
部分测试 VGA 显示图片见实验结果分析

四、实验结果分析

流水线 CPU 实验的物理验证类似单周期 CPU，主要看基本指令能否实现？寄存器数值是否根据对应指令正确修改？以及流水线各阶段能否连续、按序更新 PC 以及执行指令。

1	PC	Machine	Code Basic	Result
2	0x0	0x00100093	addi x1 x0 1	x1 = 0x1
3	0x4	0x00100113	addi x2 x0 1	x2 = 0x1
4	0x8	0x00100193	addi x3 x0 1	x3 = 0x1
5	0xc	0x00100213	addi x4 x0 1	x4 = 0x1
6	0x10	0x00802283	lw x5 8(x0)	x5 = 0x80000000
7	0x14	0x00108333	add x6 x1 x1	x6 = 0x2
8	0x18	0x0020C3B3	xor x7 x1 x2	x7 = 0x0
9	0x1c	0x40110433	sub x8 x2 x1	x8 = 0x0
10	0x20	0x05C02483	lw x9 92(x0)	x9 = 0xFFFFFFFF
11	0x24	0x00327533	and x10 x4 x3	x10 = 0x1
12	0x28	0x00502223	sw x5 4(x0)	mem[1] = 0x80000000
13	0x2c	0x005325B3	slt x11 x6 x5	x11 = 0x0
14	0x30	0x0AA3C613	xori x12 x7 170	x12 = 0xAA
15	0x34	0x0012D6B3	srl x13 x5 x1	x13 = 0x40000000
16	0x38	0x00147713	andi x14 x8 1	x14 = 0x0
17	0x3c	0x0034E7B3	or x15 x9 x3	x15 = 0xFFFFFFFF
18	0x40	0x00A50833	add x16 x10 x10	x16 = 0x2
19	0x44	0x0085C8B3	xor x17 x11 x8	x17 = 0x0
20	0x48	0x00402903	lw x18 4(x0)	x18 = 0x80000000
21	0x4c	0x004629B3	slt x19 x12 x4	x19 = 0x0
22	0x50	0x0016DA13	srli x20 x13 1	x20 = 0x20000000
23	0x54	0x00677AB3	and x21 x14 x6	x21 = 0x0
24	0x58	0x40128B33	sub x22 x5 x1	x22 = 0x7FFFFFFF
25	0x5c	0x00150B93	addi x23 x10 1	x23 = 0x2
26	0x60	0x00986C33	or x24 x16 x9	x24 = 0xFFFFFFFF
27	0x64	0x00B9CCB3	xor x25 x19 x11	x25 = 0x0
28	0x68	0x0FFA7D13	andi x26 x20 25	x26 = 0x0
29	0x6c	0x00390DB3	add x27 x18 x3	x27 = 0x80000001
30	0x70	0x002A5E33	srl x28 x20 x2	x28 = 0x10000000
31	0x74	0x0AF9EE93	ori x29 x19 175	x29 = 0xAF
32	0x78	0x001A0F33	add x30 x20 x1	x30 = 0x20000001
33	0x7c	0x00802F83	lw x31 8(x0)	x31 = 0x80000000
34	0x80	0xF81FF06F	jal x0 -128	go to 0x0
35	0x84	0x00000033	add x0 x0 x0	NOP -- 统一Pipeline各阶段
36	0x88	0x00000033	add x0 x0 x0	
37	0x8c	0x00000033	add x0 x0 x0	

以上表格记录了 P_mem.coe 中的正常程序流（无冒险版本），并记录了运行后各寄存器的数值。



上图是利用 DEMO 物理验证的 VGA 显示，主要检查两个方面：

(1) 寄存器数值：

各寄存器十分符合表格中的程序流结果，对。

(2) 流水线各阶段指令：

上图可以看到 IF、ID、Ex 的 PC 值（Mem、WB 未导出），但是可以简单推出此时 PC-Mem = 0x84，PC-WB = 0x80。从而验证了各阶段的 PC 值都是正确更新的，达到了 PipelineCPU 的实验要求。

0x80	0xF81FF06F	jal x0 -128	go to 0x0
0x84	0x00000033	add x0 x0 x0	NOP -- 统一Pipeline各阶段
0x88	0x00000033	add x0 x0 x0	
0x8c	0x00000033	add x0 x0 x0	

五、讨论与心得

相较于折磨的单周期 CPU 来说，流水线 CPU 也很折磨（特别是还要对官方 pdf 和 PPT 进行纠错。。）。但是，从实现的总体过程来说还是能够让自己比较好地理解 Pipeline 的运行原理和过程。

（Lab5-04 的结果分析会比较详细）